

What a second board changed in the AArch32 port

The AArch32 port was written for one board — a Raspberry Pi Zero 2 W, four Cortex-A53 running in 32-bit mode — and then, in step 3, asked to carry a second: the **Luckfox Lyra B**, a Rockchip RK3506 with three Cortex-A7 and a Cortex-M0.

This document is the record of what that second board actually changed, and why. It is not a summary of the Lyra; it is a list of the places where one board had been mistaken for the architecture.

1. The shape of the repository

`BOARD` already existed, with two values that were the same silicon (`zero2w`, `rpi3b`). It now selects between two genuinely different SoCs:

```
micro-os-plus-iii-aarch32/
├─ include/                    ARMv7-A, every board
│ └─ cmsis_device.h  exception_handler.hpp  hw_result.hpp  mmu.hpp
│ └─ semihosting.hpp
│ └─ port_ctx.hpp      NEW — the saved-context frame
│ └─ timer_arm.hpp     MOVED UP — CP15 generic-timer accessors
│ └─ cmsis-plus/...    the port contract headers
├─ src/                 ARMv7-A, every board
│ └─ context_switch.cpp exception_handler.cpp  handlers.cpp
│ └─ smp_secondary.cpp
│ └─ rtos/os-core.cpp   MOVED UP — the port's half of the scheduler
└─ boards/
  └─ rpi-zero-2w/       BCM2837, 4× Cortex-A53
  └─ luckfox-lyra/     RK3506, 3× Cortex-A7
    └─ src/{mmu,port_sys,smp,timer_arm}.cpp  startup.S
      └─ rtos/port_isr.cpp
    └─ include/{gic,led,osal,rk3506,smp,uart}.hpp
    └─ linker.ld  linker-qemu-virt.ld
    └─ openocd.cfg  write_board.sh
```

The Cortex-M0 is not a fourth CPU and is not in this tree. It is a different ISA outside the A7 coherency domain: it cannot take an IPI as a peer and the scheduler cannot place a thread on it. The **board** is multi-core; the **SMP cluster** is the three identical A7s. That is why the Lyra is a board directory rather than an architecture, and why `OS_NCPU` is 3.

The predecessor project stated the same fact by hard-coding `#define OS_NCPU 3` in `include/cmsis-plus/rtos/port/os-c-decls.h` — a board fact written into an architecture header. Here it is `NCPU 3` in the board's branch of `test/CMakeLists.txt`, and the header is shared unchanged.

2. Three files moved up, because one copy cannot drift

`src/rtos/os-core.cpp`

The port's half of the scheduler existed as two per-board copies, 575 and 486 lines. They were not two implementations of one design; they were the same file at two different ages, and the older one was missing fixes:

the shared file has	the older copy had
<code>vfp[32]</code> and <code>vfp_hi[32]</code> — d0–d31 saved	only d0–d15, so a thread preempted mid-NEON is silently corrupted
<code>reschedule()</code> defers when this core holds the kernel spinlock	switches anyway, stranding owner/depth on the outgoing stack
<code>switch_stacks()</code> clears owner/depth before the lock word	clears the lock first — a two-instruction window in which another core's critical section is wiped, and every core then spins forever
frame offsets derived with <code>offsetof(ctx_t, cpsr)</code>	the literals <code>34</code> and <code>200</code>
<code>lock_state</code> is <code>volatile</code>	not volatile
arrays sized <code>[OS_NCPU]</code>	sized <code>[3]</code>

Keeping both copies would have meant porting each of those fixes twice, in perpetuity, with nothing to notice when one was missed. There is one copy.

`include/timer_arm.hpp`

Every accessor in it is a CP15 instruction. That is a fact about ARMv7-A, not about a board.

`include/port_ctx.hpp` (new)

`ctx_t` and the offsets derived from it. It is shared because the frame is built by this port's own assembly (`handlers.cpp`, `context_switch.cpp`) and read by both the shared scheduler and each board's ISR, and all four have to agree.

3. What stayed per board, and the reason in each case

file	why it cannot be shared
<code>src/rtos/port_isr.cpp</code>	The BCM2837 has no GIC : the taken interrupt is read from a per-core local "IRQ source" register and the IPI is cleared through a mailbox. The RK3506 has a GIC-400: the interrupt comes from the CPU interface's IAR and every one of them needs an EOI with the same value. The dispatch is the interrupt controller.
<code>src/timer_arm.cpp</code>	The Pi has to measure its counter rate against the BCM2835 1 MHz system timer, because its <code>CNTFRQ</code> reports 19.2 MHz while the counter actually increments at ~1 MHz. The RK3506's <code>CNTFRQ</code> is programmed by the miniloader and is honest.
<code>src/smp.cpp</code> , <code>src/startup.S</code>	Releasing the secondaries: the Pi writes an entry address into the VideoCore armstub's mailbox-3 spin table (and <code>__smp_spin</code> + <code>SEV</code> under QEMU); the Lyra publishes an entry into an SRAM mailbox and clears the core's reset bits in the CRU.
<code>src/mmu.cpp</code> , <code>linker.ld</code>	Different memory maps — DRAM at <code>0x0001_0000</code> vs <code>0x0020_0000</code> , peripherals at <code>0x3F00_0000</code> vs <code>0xFF00_0000</code> .
<code>include/uart.hpp</code>	PL011 vs a DesignWare 16550.
<code>include/led.hpp</code>	A plain SoC GPIO vs GPIO1_A0 behind three CRU gates and a pinmux. On the Lyra this header also replaces thirteen verbatim copies of the same ungate/de-reset/mux sequence, one in each of the predecessor project's test directories.
<code>PORT_RAM_BASE</code> / <code>PORT_RAM_END</code> in <code>include/smp.hpp</code>	The DRAM window a stack pointer is checked against before the port switches to it.

4. Seven portability defects the second board exposed

These were all in code that had been shared for some time. None of them was visible while every board had four cores and the same drivers.

1. Every application compiled the SD and USB drivers. `uos_add_app()` linked `micro-os-plus::devices` whenever the target existed, so the `LIBRARIES` argument each test already passed was dead. On the Pi this was invisible. On the Lyra it is a hard failure — `usb_dwc2.cpp` includes `bcm2837.hpp` — which is how it was found. The caller now names what it links.

2. `wn[OS_NCPU] = {"w0", "w1", "w2", "w3"} (smp_test2)` — *too many initializers* at `OS_NCPU` 3. Names are generated now.

3. `port_cpu_id() & (OS_NCPU-1)` and `port_cpu_id() & 3u` — a modulo only while the core count is a power of two. At three cores, core 2 lands in slot 0 and the work-distribution report is wrong **without failing**. Replaced by `cpu_slot()`, which clamps.
4. **Six tests named cores 1, 2 and 3 by hand** to wait for the secondaries, print the join line and fold it into the verdict — reading one element past the end of `g_core_stage[OS_NCPU]`. Replaced by three shared helpers: `test_wait_secondaries()`, `test_secondaries_joined()`, `test_join_summary()`.
5. `g_core_produced[4]`, `g_core_consumed[4]`, `g_core_yields[4]` (`smp-pro-cons-test`) — fixed-size per-core tallies, printed as `c0..c3` and checked as `c0..c3`. Now `[OS_NCPU]`, printed and checked by loop (`report_per_core()`).
6. `boot_core3()` in the shared `src/smp_secondary.cpp` writes `g_core_stage[3]`. Guarded with `#if OS_NCPU > 3`.
7. `{0, 0, 0, 0}` initialisers for `[OS_NCPU]` arrays in `os-core.cpp`. Now `{}`.

Everything above is in the **shared** kernel and test tree, so the fixes apply to AArch64 as well. Both ports were re-run afterwards: 11 passed, 1 skipped (`usb_test`, no USB device model), 0 failed, on each.

5. Build-system changes

```
cmake -S . -B build-lyra -DBOARD=luckfox-lyra \
-DMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/arm-none-eabi.cmake
```

knob	zero2w / rpi3b	luckfox-lyra
<code>UOS_BOARD_DIR</code>	<code>rpi-zero-2w</code>	<code>luckfox-lyra</code>
CPU flags	<code>-mcpu=cortex-a53 -mfpu=neon-fp-armv8</code>	<code>-mcpu=cortex-a7 -mfpu=neon-vfpv4</code>
SoC define	<code>SOC_BCM2837</code>	<code>SOC_RK3506</code>
extra library	<code>micro-os-plus::soc-bcm2837</code>	—
<code>NCPU</code>	4	3
<code>hwd</code> linker script	<code>linker.ld</code>	<code>linker.ld</code>
<code>qemu</code> linker script	<code>linker.ld</code>	<code>linker-qemu-virt.ld</code>
test applications	all twelve	the seven SoC-neutral ones

Two of those rows are new shapes, not just new values:

- **The linker script is now per variant, not only per board.** The Lyra's emulated image is linked for a different machine than its hardware image.

- `QEMU_BUILD` is now defined for the `qemu` variant. It does not mean "not hardware" — it means *this image runs under the emulator*, which for a board QEMU does not model is a different machine with a different console, a different interrupt controller and a different DRAM base.

Why seven applications and not twelve

`sd_test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test` and `usb_test` link `micro-os-plus::devices`, whose backends are written against the BCM2837's EMMC and DWC2. The RK3506 has neither at those addresses. Until those drivers grow a second backend the Lyra builds the subset that needs only CPUs, a timer and a console — which is also the only subset its predecessor project ever had.

6. QEMU: `-M virt`, and no shim

The Pi runs under `-M raspib` through a 20-line AArch64 stub, because QEMU starts its Cortex-A53 cores in AArch64 and the port is 32-bit.

The Lyra needs no shim: a Cortex-A7 has no AArch64 state to drop out of. It runs under `-M virt`, the generic machine — PL011 console at `0x0900_0000`, GICv2 at `0x0800_0000`, DRAM at `0x4000_0000`.

That is a deliberate choice, not a convenience. QEMU models none of the RK3506's own blocks — the CRU, the SRAM mailbox, the Rockchip USB gadget, the Cortex-M0. Emulating the Lyra would mean emulating the parts of it that are least like a Cortex-A7. So **the emulated runs cover the SoC-neutral subset only**, and everything specific to the board is hardware-only.

Status of this path

The `-M virt` images **build**. They do not run yet: `src/mmu.cpp` still maps DRAM at `0x0020_0000` and the peripheral window at `0xFF00_0000`, and `gic.hpp` still carries the RK3506's GIC bases. `uart.hpp` and `led.hpp` already have their `QEMU_BUILD` paths, and `linker-qemu-virt.ld` and the `PORT_RAM_*` window are in place. The MMU table and the GIC bases are what remains.

7. Hardware: what the shared runner had to learn

`test/run-hw.sh` in the kernel repository drives every hardware session. It had three things hard-coded that were the Pi's, not the architecture's:

was	is
target names spelled <code>bcm2837.cpu\$core</code>	<code>UOS_HW_TARGET_FMT</code> , a printf format — <code>rk3506.a7.%d</code> for the Lyra
every core 0..NCPU-1 halted, loaded and resumed	<code>UOS_HW_CORES</code> , the indices that are debug targets <i>at load time</i>
halt → load → resume, nothing in between	<code>UOS_HW_PRELOAD</code> , Tcl run after the halt and before <code>load_image</code>

and it gained a third resume mode, `entry` — resume at the entry point touching no register — beside `pc` (AArch64) and `cpsr` (AArch32 on the Pi).

Each of the three is a real property of the RK3506, not a convenience:

- **Only core 0 is examinable when the image is loaded.** Cores 1 and 2 are in the BootROM until the kernel clears their CRU reset bits, which is why `openocd.cfg` declares them `-defer-examine`. Halting a target that has not been examined fails.
- **There is no `__smp_spin`.** The Pi's secondaries park in a spin table the runner zeroes after the load so a core that reaches its parking loop before core 0 clears `.bss` cannot jump to a stale entry. The Lyra's are released through an SRAM mailbox instead, so `UOS_HW_SPIN_WORDS=0` and the stage is skipped entirely.
- **The MMU and caches are ON** when the miniloader hands core 0 over. `load_image` writing through a dirty cache leaves DRAM holding something other than the image, so `SCTLR.{M,C,I}` are cleared and the I-cache, branch predictor and TLB invalidated first — the same sequence the board's own `write_board.sh` has always used.

The Pi's generated script is unchanged by all of this, byte for byte apart from `[format {bcm2837.cpu%d} $core]` where it said `bcm2837.cpu$core`.

`boards/luckfox-lyra/openocd.cfg` and `write_board.sh` are one copy each. The predecessor project had thirteen of each, one per test directory.

```
BOARD=luckfox-lyra test/hw.sh list
BOARD=luckfox-lyra test/hw.sh smp_test0
```

8. What is not done

- The `-M virt` MMU map and GIC bases (above).
- The eight RK3506-specific tests the predecessor project had — `smp_test5`, `smp_test6`, `smp_test7`, `smp_test_int` and `smp_test_int2..5` — which exercise GIC priority and preemption, the Rockchip USB gadget and the Cortex-M0 mailbox. They are hardware-only by nature. `smp_test0` — `smp_test4` exist in both projects under the same names but are **different tests**; the unified ones are in use.

- Hardware validation on a Lyra. Nothing in this document claims a Lyra has run.